Software Quality Engineering (SQE) — Mids Notes

Lecture 1 se Lecture 4 tak · Roman Urdu mein · asaan explanation + real-world examples

Kaise parhna hai: Har topic mein pehle *Matlab* (simple explanation), phir *Example* (asal zindagi se), aur aakhir mein *Yaad rakho* (exam point) diya hai. Slides ka har point cover kiya gaya hai, bas usay samajhne layak bana diya hai. Technical words (jaise defect, failure, regression) English mein hi rakhe hain taake paper mein wohi likh sako.

Lecture 1: Software Quality & Quality Assurance

1.1 Software Quality kya hai?

Definition: Software quality woh degree hai jahan tak koi software product apni stated functional aur non-functional requirements ko poora kare, aur saath mein un implicit expectations par bhi poora utre jo ek professionally bane hue software se ki jati hain.

Asaan lafzon mein: Sirf yeh dekhna kaafi nahi ke software *chal raha hai ya nahi*. Quality ka matlab hai: sahi kaam kare (correctness), har dafa ek jaisa kaam kare (consistency), istemal mein aasan ho (ease of use), tez ho (efficiency), aur aage chal kar isay theek aur extend karna mushkil na ho (maintainability).

Quality teen cheezon se judge hoti hai:

Cheez	Matlab	Example
Explicit Requirements	Jo specification mein seedha likha hai	User email aur password se login kar sake
Documented Standards	Coding, design aur process ke woh standards jin par team ne commit kiya	Har code review hoga, naming convention follow hogi
Implicit Expectations	Jo likha nahi hota magar sab maan kar chalte hain	Login page jaldi khule, password screen par dikhe nahi, app bila wajah crash na ho

Real-world example: WhatsApp. Requirement thi 'message bhejna'. Magar kisi ne yeh alag se nahi likha ke message secure ho, delivery tick aaye, aur app raat ko crash na kare. Yeh sab implicit expectations hain. Agar yeh poori na hon to user kahega 'app achi nahi hai', chahe message bhej bhi raha ho.

Yaad rakho: Quality = explicit requirements + documented standards + implicit expectations.

1.2 Software Quality ke do views

Quality ko do angles se dekha jata hai: product *kitni achi tarah soch kar banaya gaya* aur *kitni imaandari se banaya gaya*.

	Quality of Design	Quality of Conformance
Matlab	Code likhne se pehle requirements, architecture aur specifications kitni achi tarah conceive hui	Bana hua product approved design aur specs se kitna milta hai
Key question	Kya design sahi problem ko sahi tareeqe se solve kar raha hai?	Kya implementation bina shortcuts aur unresolved defects ke wohi deliver kar rahi hai jo design mein tha?
Kab dekhte hain	Coding se pehle	Coding ke dauran aur baad mein

Real-world example (ghar): Architect ne ghar ka naksha banaya, yeh *Quality of Design* hai. Mistri ne naksha dekh kar ghar banaya, yeh *Quality of Conformance* hai.

- Agar naksha hi ghalat tha (kamray ki jagah nahi, hawa ka rasta nahi), to mistri chahe kitna bhi perfect kaam kare, ghar achha nahi banega. Yeh **design ki kharabi** hai.
- Agar naksha bohat achha tha magar mistri ne cement kam daal diya aur deewar tehri bana di, to yeh **conformance ki kharabi** hai.

Software mein bhi aisa hi hota hai: requirements/architecture ghalat hon to design quality kharab, aur code design se hat kar likha jaye ya defects chhor diye jayein to conformance quality kharab.

Yaad rakho: Design = 'kya aur kaise banana hai' ki quality. Conformance = 'jo socha tha wohi bana ya nahi' ki quality. Dono chahiye.

1.3 Software Quality kyun zaroori hai? (5 wajahein)

1. **Customer Trust & Retention:** Reliable software se user khush rehta hai aur churn (users ka app chhor kar chale jana) kam hota hai. *Example:* Agar koi mobile wallet app payment ke waqt baar baar fail ho, user kisi doosri app par shift ho jata hai.
2. **Lower Cost of Ownership:** Kam defects matlab kam rework, kam support calls, kam emergency fixes. *Example:* Raat 3 baje production bug ke liye poori team ko bulana bohat mehnga padta hai, jabke pehle hi test ho jata to sasta pad jata.

3. **Brand & Reputation:** Public failures aur outages se credibility jaldi giri hai. *Example:* July 2024 mein CrowdStrike ke ek faulty update ki wajah se duniya bhar mein airlines, banks aur hospitals ke computers prabhavit hue. Ek software galti se company ka naam kharab hua.
4. **Safety & Compliance:** Kai fields mein qanoon verified aur dependable software maangta hai. *Example:* Hospital ki machines, jahaz ka software, banking systems.
5. **Competitive Advantage:** Do milti julti apps mein jo kam crash karti hai, log usi ko rakhte hain. Consistent quality product ko market mein alag karti hai.

1.4 Dimensions of Software Quality (6 dimensions)

Yaad karne ka trick: **F-R-U-E-M-P** (Functionality, Reliability, Usability, Efficiency, Maintainability, Portability). Sab ko ek **ATM machine** ke example se samjhte hain:

Dimension	Matlab	ATM Example
Functionality	Required kaam sahi aur mukammal kare, har specified feature poora ho	ATM jitna paisa maango utna hi nikaale, balance sahi dikhaye
Reliability	Waqt ke saath bina unexpected failure, crash ya data loss ke chale	Din bhar chalta rahe; transaction beech mein fail ho to paisa na kate
Usability	Intended users asani se seekh, samajh aur chala saken	Bujurg shakhs bhi screen ke steps dekh kar paisa nikaal le
Efficiency	CPU, memory, storage jaise resources ka economical istemal	Purani, kam power wali machine par bhi tez chale
Maintainability	Reasonable effort mein modify, correct aur extend ho sake	Naye bank ka card support add karna asaan ho
Portability	Alag platforms/environments par transfer ho kar reliably chale	Same banking software alag models ki ATM machines par chale

1.5 Common Software Quality Models

Models ka kaam hai quality ko define aur measure karne ke liye ek **common vocabulary** dena (jaise nap tol ke liye 'meter' aur 'kilo' ek standard hain).

1. **McCall's Quality Model:** Sabse purane models mein se ek. Quality factors ko teen groups mein baant'ta hai: **Product Operation** (software kaise kaam karta hai), **Product Revision** (change karna kitna asaan), **Product Transition** (naye environment mein

jana). *(Extra, agar teacher ne bataya ho: Operation mein correctness, reliability, efficiency, integrity, usability; Revision mein maintainability, flexibility, testability; Transition mein portability, reusability, interoperability.)*

2. **ISO/IEC 25010:** Modern international standard jo **8 quality characteristics** define karta hai (reliability, security waghera). Iski poori list Lecture 2 mein hai.
3. **CMMI (Capability Maturity Model Integration):** Yeh *product* ki nahi balke *development process* ki maturity rate karta hai. *(Extra: 5 levels hote hain, Initial se Optimizing tak.)*

Example: McCall aur ISO 25010 batate hain 'product kaisa hona chahiye', jabke CMMI batata hai 'company ka kaam karne ka tareeqa kitna organized hai'. Jaise ek restaurant ka khana (product) aur uska kitchen system (process) alag cheezein hain.

1.6 Quality Assurance (QA) kya hai?

Definition: QA ek **proactive, process-oriented** discipline hai jo quality ko development process ke andar hi *build* karta hai, taake defects shuru se hi kam paida hon.

Asaan lafzon mein: QA tayyar shuda software mein ghalti nahi dhoondta. Woh us *process* ko theek karta hai jo software banata hai, standards, training, reviews aur audits ke zariye, taake ghaltiyan banein hi kam.

> **Slide ki line:** 'QA builds the road so fewer accidents happen; QC inspects the cars driving on it.' Yaani QA sadak achi banata hai taake hadsay kam hon, aur QC sadak par chalti gariyon ko check karta hai.

Real-world example: Ek bottling factory mein QA hai: hygiene ke SOPs banana, workers ki training, machines ka regular audit. QC hai: bani hui bottle uthakar dekhna ke seal theek hai ya nahi.

1.7 QA ke Objectives (4)

1. **Process-Focused:** Yeh behtar karta hai ke software *kaise banta hai*, sirf yeh nahi ke *kya deliver hua*.
2. **Proactive by Nature:** Defects ko introduce hone se *pehle* hi rokna.
3. **Standards & Reviews:** Guidelines, audits aur structured reviews par depend karna.
4. **Continuous Improvement:** Seekhe huay sabaq (lessons learned) wapas process mein daalna.

Sab milkar quality ko process ka built-in hissa banate hain, baad mein lagaya hua add-on nahi.

1.8 QA vs QC

Quality Assurance (QA)	Quality Control (QC)
Prevention-focused	Detection-focused
Development *process* ko improve aur monitor karta hai	Asal *product/deliverable* ko examine karta hai
Poore lifecycle mein apply hota hai	Zyada tar implementation ke dauran aur baad mein
Standards, training, process audits shaamil	Testing, inspections, reviews shaamil
Goal: defects ko introduce hone se rokna	Goal: jo defects maujood hain unhein dhoondna aur hatana

Dono saath kaam karte hain: QA kam karta hai ke kitne defects paida hon; QC pakadta hai jo QA na rok saka.

Example: Code review ke rules banana aur team ko training dena = QA. Bani hui app ko chala kar bugs dhoondna = QC.

1.9 QA Process in Practice (5 steps ka loop)

1. **Define Standards:** Coding, design aur documentation ki guidelines tay karo.
2. **Train the Team:** Un standards mein team ki skill banao.
3. **Conduct Reviews:** Requirements, design aur code ka peer review karo.
4. **Audit the Process:** Independently check karo ke standards follow ho rahe hain.
5. **Improve Continuously:** Findings ko agle cycle mein wapas feed karo.

Yeh loop hamesha chalta rehta hai. QA ek dafa ka checklist nahi, balke musalsal chalne wala cycle hai.

Example: Team ne rule banaya ke har function par comment likhna hai (step 1), naye members ko sikhaya (2), pull request par review hua (3), mahine ke end par manager ne dekha ke rule follow hua ya nahi (4), aur dekha gaya ke rule se kaam slow ho raha hai to rule mein tabdeeli hui (5).

1.10 Defect Management Strategy: Teen Layers of Defense

Software mein defects ko teen levels par handle karte hain:

Layer	Kaam	Zindagi se example (sehat)
Defect Prevention	Defects ko introduce hone se rokna, root cause theek karke	Vaccine lagwana taake bimari hi na ho
Defect Reduction	Jo defects ban chuke hain unhein release se *pehle* dhoondna aur hatana	Bimari ho jaye to doctor se check-up aur ilaaj
Defect Containment	Jo defect production tak pahunch jaye uska nuqsan limit karna	Bimari phail jaye to patient ko quarantine karna taake doosron tak na pahunche

Slide ki line: Prevention kam karta hai kitne defects paida hon; Reduction pakadta hai jo nikal gaye; Containment unka nuqsan limit karta hai jo phir bhi reh gaye.

1.11 Pillar 1: Defect Prevention

Matlab: Defects ko tab hi rok dena jab woh banne wale hon, unki **root causes** khatam karke. Sirf symptoms (nishaniyan) theek karna kaafi nahi.

Core approaches:

- Past defects par **Root Cause Analysis (RCA)**
- Requirements aur design ke formal reviews
- Coding standards aur design guidelines
- Common ghalti wale areas par targeted training

Sabse zyada important kyun? Requirements stage par roka gaya defect production mein pakde gaye defect ke muqable mein bohat sasta padta hai. Jitna pehle roko, utna sasta aur safe fix. Isiliye prevention poori strategy ka sabse high-leverage investment hai.

Root Cause Analysis (RCA)

RCA symptom se aage dekh kar *asal wajah* dhoondta hai, taake wohi ghalti poore project mein dobara na ho.

Step	Kaam
1. Identify the Defect	Exactly kya ghalat hua aur kahan, yeh note karo
2. Trace to Origin	Defect ko peeche follow karo us stage tak jahan woh introduce hua
3. Categorize the Cause	People, process ya tooling ka masla?

Step	Kaam
4. Correct & Monitor	Process theek karo aur dekho ke dobara to nahi aa raha

Common causes: unclear requirements, training ki kami, inadequate reviews, tooling gaps.

Example: Login form mein user ne 3 character ka password daal diya aur system ne accept kar liya. *Symptom:* weak password. Sirf us ek form ko theek karna fix nahi hai. RCA se pata chala ke *requirement mein password length likhi hi nahi thi* (unclear requirements). Asal fix: requirements template mein 'validation rules' ka section lazmi kar do, taake aage har form mein yeh ghalti na ho.

Defect Prevention Techniques (6)

1. **Requirement Clarity Sessions:** Coding se pehle client ke saath baith kar requirements saaf karna.
2. **Design & Code Reviews:** Doosre developers kaam check karte hain.
3. **Pair Programming:** Do developers ek saath, ek code likhta hai doosra saath saath review karta hai.
4. **Static Analysis Tools:** Tools jo code chalaye baghair hi ghaltiyan dhoondte hain (linters).
5. **Reusable Templates & Components:** Pehle se test shuda cheezein dobara istemal karna, naye bugs kam.
6. **Process Maturity (CMM/CMMI):** Mazboot, organized process.

Har technique process ke alag stage ko target karti hai: shuru mein intent clear karne se le kar tools se standards enforce karne tak.

Cost of Fixing Defects (Chart)

Jitna der se defect pakda jaye, utna mehnga aur disruptive hota hai fix. Slide 17 ke mutabiq relative cost:

Stage	Relative Cost
Requirements	1.0
Design	3.0
Coding	7.0
Testing	15.0

Stage	Relative Cost
Production	30.0

Example: Requirement mein 'GST 17% hoga' likhna bhool gaye. Agar yeh meeting mein pakda jaye to bas ek line badalni hai (cost 1). Agar yeh app live hone ke baad customers ke bills mein ghalat nikle to refund, support, patch aur maafi sab karna padega (cost 30). Yehi prevention ka sabse bara argument hai.

1.12 Pillar 2: Defect Reduction

Matlab: Jo defects development mein aa chuke hain unhein jitna jaldi ho sake aur *customer tak pahunchne se pehle* dhoondna aur hatana. Yeh wahan se shuru hota hai jahan prevention khatam hoti hai, kyunke maan liya jata hai ke kuch defects phir bhi nikal jayenge.

Core approaches: Multiple levels par structured testing; verification aur validation; peer code reviews aur walkthroughs; static aur dynamic code analysis.

Verification vs Validation

Verification	Validation
'Are we building the product **right**?'	'Are we building the **right** product?'
Check karta hai ke process ka har stage apni specification ke mutabiq hua	Check karta hai ke finished product customer ki asal zarooraton ko poora karta hai
Reviews, walkthroughs, inspections se	Real-world use cases ke against testing se

Example (darzi): Customer ne kurta maanga. Darzi ne naap ke mutabiq perfect silai ki, magar customer ko *shalwar kameez ka poora suit* chahiye tha. Silai sahi thi (verification pass), magar galat cheez bani (validation fail). Doosri taraf, aisa kurta jo customer ko pasand to aaya magar andar se silai kachi thi, woh validation pass magar verification fail.

Yaad rakho: Verification bina validation = galat cheez perfectly ban gayi. Validation bina verification = andar ke defects miss ho gaye. Dono zaroori.

Testing Levels (5)

Left se right tests ka scope chhote function se poore customer experience tak badhta jata hai.

1. **Unit Testing:** Individual functions aur components. *Example:* calculateTotal() function ko akele test karna.

2. **Integration Testing:** Combined modules ka aapas ka interaction. *Example:* Cart module aur Payment module ka milkar sahi kaam karna.
3. **System Testing:** Poori end-to-end application. *Example:* Login se le kar order place karne tak.
4. **Acceptance Testing:** Customer requirements ke against validate karna. *Example:* Client khud app chala kar kehta hai 'haan, yehi chahiye tha'.
5. **Regression Testing:** Confirm karna ke naye changes ne purani cheezein nahi tori. *Example:* Naya discount feature lagane ke baad dekhna ke checkout abhi bhi chal raha hai.

Tracking Defects to Closure

Defect milne ke baad woh ek tracked lifecycle se guzarta hai taake kuch bhoola na jaye:

New → Assigned → Fixed → Verified → Closed

- **Common tools:** Jira, Bugzilla, Azure DevOps (defects log, prioritize, assign aur monitor karne ke liye).
- **Useful metrics: Defect density** (code ki ek unit mein kitne defects) aur **defect leakage rate** (kitne defects agle stages mein escape kar gaye).

Example: Tester ne Jira mein bug dala (New), manager ne developer ko diya (Assigned), developer ne theek kiya (Fixed), tester ne dobara chala kar confirm kiya (Verified), aur ticket band ho gaya (Closed).

1.13 Pillar 3: Defect Containment

Matlab: Jo defect testing se bhi nikal kar production mein pahunch jaye, uska nuqsan limit karna: woh isolate ho, jaldi detect ho, aur poore system mein cascade (phail) na kare. Koi process sab kuch nahi pakad sakta, isliye containment maanta hai ke kuch defects users tak pahunchenge, aur impact ko chhota, mukhtasar aur recoverable banane par focus karta hai. Yeh **last line of defense** hai.

Core approaches: Modular architecture jo failures ko isolate kare; graceful error/exception handling; real-time monitoring aur alerting; fast, reliable rollback.

Containment Techniques (6)

Technique	Matlab	Real-world example
Fault Isolation	Modular design, ek module ki failure doosron mein na phaile	Ghar mein alag alag circuit breakers: ek kamray ka short circuit poore ghar ki bijli nahi uraata

Technique	Matlab	Real-world example
Exception Handling	Graceful error handling, hard crash se bachata hai	App crash hone ki jagah likhe 'Network masla hai, dobara koshish karein'
Monitoring & Logging	Real-time visibility, masla hote hi pata chale	Gari ka dashboard warning light
Redundancy & Failover	Ek component fail ho to backup uski jagah le le	Load shedding mein UPS ya generator ka automatic on ho jana
Phased / Canary Releases	Nayi release pehle chhote user group ko	Naya feature pehle 5% users ko dena, masla ho to baqi bach jayein
Rollback Mechanisms	Last stable version par jaldi wapas ana	Naya update kharab nikla to phone ko purane version par wapas le jana

1.14 Lecture 1 Summary

- **Software Quality:** Functional, performance aur implicit standards ko poora karna jo ek professional product se expected hain.
- **Quality Assurance:** Proactive, process-level discipline jo poore lifecycle mein quality high rakhta hai.
- **Prevention → Reduction → Containment:** Defect ko source par roko, jo bacha release se pehle pakdo, aur jo phir bhi reh jaye uska asar limit karo.

Mids ke possible sawal (Lecture 1):

1. Software quality define karein aur iske teen components batayein.
2. Quality of Design aur Quality of Conformance mein farq example ke saath likhein.
3. Software quality ki 6 dimensions explain karein.
4. QA aur QC mein farq likhein (table banayein).
5. QA process ke 5 steps likhein.
6. Defect management ki teen layers explain karein.
7. RCA kya hai? Iske 4 steps example ke saath likhein.
8. Verification aur Validation mein farq likhein.
9. Testing ke 5 levels batayein.
10. Defect containment ki techniques likhein.

Lecture 2: Software Quality Attributes & Quality Engineering

Yeh lecture do hisson mein hai: **Part 1: Quality Attributes** aur **Part 2: Quality Engineering**.

Part 1: Software Quality Attributes

2.1 Software Quality (dobara, teen nazariye)

Software quality woh degree hai jahan product apni specified requirements poori kare aur users ki zarooraton aur umeedon ko satisfy kare. Iske teen views hain:

View	Sawal	Example
Conformance to requirements	Kya woh wohi karta hai jo specify kiya gaya tha? (functional correctness)	Spec mein likha 'transfer ho', aur transfer ho raha hai
Fitness for use	Kya yeh asal conditions mein users ki asal zaroorat poori karta hai?	App lab mein theek chalti hai, magar kamzor internet wale gaon mein bhi chale
Structural quality	Kya code achhe se bana hai: maintainable, testable, secure?	Saaf, organized code jise naya developer samajh sake

2.2 Quality Attributes kya hain?

- **Functional requirements** batate hain system **KYA** karta hai.
- **Quality attributes** batate hain system woh kaam **KITNI ACHI TARAH** karta hai.

Example (bank transfer):

- Functional: 'Customer ek account se doosre account mein paisa transfer kar sake.'
- Quality attribute: 'Transfer 2 second se kam mein complete ho, system 99.99% waqt available ho, aur sab encrypted ho.'

In ko **non-functional requirements (NFRs)** ya **'-ilities'** bhi kehte hain (reliability, usability, scalability...). Yeh software **architecture ke main drivers** hain aur jab yeh missing hon to users ko sabse zyada mehsoos hote hain.

Example: Restaurant mein khana (functional) mil gaya, magar 2 ghante baad aaya (performance attribute fail). Khana mila, magar service kharab thi, isliye customer naraz.

2.3 ISO/IEC 25010 Model: 8 Characteristics

#	Characteristic	Matlab	Example
1	**Functional Suitability**	Sahi cheezein, sahi tarah kare	Calculator 2+2=4 de
2	**Performance Efficiency**	Load ke neeche speed aur resource use	Sale ke din bhi site tez khule
3	**Compatibility**	Doosre systems ke saath coexist aur data exchange	Alag apps aapas mein data share kar saken
4	**Usability**	Aasan, seekhne mein asaan, accessible	Naya user bina manual ke app chala le
5	**Reliability**	Jab zaroorat ho bina failure ke chale	Banking app hamesha available
6	**Security**	Data ki hifazat aur ghalat istemal rokna	Doosra banda aap ka account na khol sake
7	**Maintainability**	Modify aur fix karna asaan	Bug 1 din mein theek ho jaye
8	**Portability**	Alag environments mein move ho sake	Windows se Linux par bhi chale

2.4 Runtime Attributes (chalte hue nazar aati hain)

Yeh woh attributes hain jo users system chalte waqt *observe* kar sakte hain:

Attribute	Sawal	Measure kaise hota hai
Performance	Kitna tez respond karta hai?	Response time, throughput, resource usage
Security	Kya data protected hai?	Confidentiality, integrity, authentication
Availability	Kya zaroorat ke waqt up hai?	Uptime %, recovery time, failover
Usability	Kya users aasani se kamyab ho paate hain?	Learnability, error rate, accessibility

2.5 Design-time Attributes (structure ke andar chhupi hoti hain)

Yeh end users ko nazar nahi aati, magar yeh tay karti hain ke system ko change, test aur move karna kitna mehnga hai (long-term cost of ownership).

Attribute	Matlab	Tactic (tareeqa)
Maintainability	Defects dhoondna aur theek karna kitna asaan	Modular design, low coupling
Testability	Behaviour ko verify karna kitna asaan	Dependency injection, test hooks
Modifiability	Naye features add ya change karne ka effort	Encapsulation, interfaces
Portability	Doosre environments mein chalne ki salahiyat	Abstraction layers, containers

Example: Do gharon ki wiring: ek mein sab taarein uljhi hui (maintainability kharab), doosre mein har switch ka alag board. Bijli dono mein aati hai (runtime theek), magar naya bulb lagana ya fault dhoondna pehle ghar mein bohat mehnga hai.

2.6 Performance & Scalability (depth mein)

Term	Matlab	Example target
Response time	Kitni jaldi jawab aata hai	API target: < 200 ms
Throughput	Kitni requests handle hoti hain	10k requests/second
Latency percentile (p95)	95% requests is time ke andar; worst 5% matter karti hain	p95 < 2 sec
CPU utilisation	CPU kitna use ho raha hai	< 70% (spikes ke liye headroom)

p95 samjho: Agar 100 users ne site kholi aur 95 ko jawab 2 second ke andar mila, to p95 = 2 sec. Average achha dikha sakta hai magar worst 5% users ka bura tajurba chhupa leta hai, isliye p95 dekhte hain.

Common tactics:

- Frequently used data ko **cache** karna
- Servers ke beech **load-balance** karna
- **Asynchronous processing** aur queues

- Database **indexing** aur query tuning
- **Horizontal scaling** (zyada nodes add karna)

Scalability: *Vertical* = ek bari machine (zyada RAM/CPU). *Horizontal* = zyada machines.

Example: Online sale (jaise 11.11 ya Eid sale) par achanak laakhon users aate hain. Vertical scaling = ek dukaan ko bara karna. Horizontal scaling = aur dukaanein (counters) khol dena. Load balancer wohi hai jo gate par khara ho kar customers ko khali counter ki taraf bhejta hai.

2.7 Security & Reliability (depth mein)

Security ka CIA triad:

- **Confidentiality:** Sirf authorised log data dekh saken. *(Chitthi sirf us ke liye jis ke naam hai.)*
- **Integrity:** Data raaste mein badla na jaye. *(Chitthi ka mazmoon kisi ne tabdeel na kiya ho.)*
- **Availability:** Jab zaroorat ho tab system mile.

Security tactics: Authentication aur authorisation, encryption in transit aur at rest, audit logging aur intrusion detection.

Reliability tactics: Fault tolerance aur redundancy, timeouts/retries/circuit breakers, health checks, graceful degradation.

Availability formula (zaroor yaad rakho):

Availability = MTBF ÷ (MTBF + MTTR)

- **MTBF** = Mean Time Between Failures (do failures ke beech ka average waqt)
- **MTTR** = Mean Time To Repair (theek karne ka average waqt)

Numerical example: Agar server har 99 ghante baad ek dafa kharab hota hai (MTBF = 99) aur theek karne mein 1 ghanta lagta hai (MTTR = 1), to Availability = 99 ÷ (99 + 1) = 0.99 = **99%**. Yaani availability barhane ke do raaste: MTBF barhao (kam kharab ho) ya MTTR ghatao (jaldi theek ho).

Graceful degradation example: Online shop ka recommendation engine down ho gaya, to poori site band karne ke bajaye sirf recommendation section chhupa do aur baqi site chalne do. **Circuit breaker:** Agar payment service baar baar fail ho rahi hai to uski taraf bar bar request na bhejo, kuch der ruk jao taake poora system na atke (jaise ghar ka MCB).

2.8 Usability & Maintainability (depth mein)

Usability	Maintainability
Learnability: Naye users jaldi kamyab hon	**Modularity:** Change ek hi jagah tak mehdood rahe
Efficiency: Kaam ke liye kam steps	**Low coupling, high cohesion:** Saaf boundaries
Error prevention: Saaf messages, undo	**Readable code:** Consistent style aur naming
Accessibility: WCAG, screen readers	**Living documentation:** Up-to-date docs aur tests

Examples: *Error prevention:* Gmail mein 'Undo Send'. *Accessibility:* Nabeena afraad ke liye screen reader support. *Low coupling:* Ek module ko badalna to doosre ko na chhedna padey. *High cohesion:* Ek module ke andar ki cheezein ek hi kaam se talluq rakhein.

2.9 Quality Attribute Scenarios (6-part format)

Quality requirement ko **testable** shakal mein likhne ka format, jisme 6 parts hote hain: **Source, Stimulus, Artifact, Environment, Response, Response Measure.**

Part	Slide ka example
Source	End user
Stimulus	1,000 concurrent searches
Artifact	Search service
Environment	Normal peak load
Response	Results return karta hai
Response Measure	95% requests 2 seconds ke andar

Kyun zaroori? Yeh ek mubham khwahish ('system tez hona chahiye') ko **measurable aur testable** requirement bana deta hai. Upar wale scenario ka matlab: jab end user normal peak load par search kare, to search service 95% requests ke results 2 second ke andar de.

Apna example (exam mein bana sakte ho): Source = Student, Stimulus = result check karna jab 5,000 students ek saath aayein, Artifact = Result portal, Environment = Result announcement ka din, Response = Result page khulta hai, Response Measure = 90% requests 3 second ke andar.

2.10 Trade-offs Between Attributes

Ek attribute behtar karne se aksar doosre ki qeemat deni padti hai. Priorities stakeholders ki zaroorat se tay hoti hain (jaise **ATAM** method se).

Tension	Kyun takrao hota hai	Typical solution	Example
Security ↔ Usability	Extra login steps users ko slow karte hain	Risk-based authentication (2FA sirf sensitive actions par)	Bank app: balance dekhne par PIN, lekin bari transfer par OTP
Performance ↔ Maintainability	Highly optimised code parhna aur badalna mushkil	Sirf measured hot spots optimise karo, baqi code saaf rakho	Sirf wohi function optimise karo jo profiler mein slow nikla
Availability ↔ Cost	Redundancy aur failover ke liye extra infrastructure	Targets business impact se tay karo (99.9% vs 99.99%)	Hospital system ko 99.99%, college ki notice board site ko 99%
Modifiability ↔ Performance	Abstraction layers extra indirection laate hain	Jahan change expected ho wahan layer lagao; critical paths benchmark karo	Bohat layers ka code flexible magar thoda slow
Portability ↔ Performance	Generic code platform-specific features use nahi kar sakta	Platform code ko interfaces ke peeche isolate karo	Cross-platform app vs native app

Yaad rakho: Koi system sab attributes mein 100% nahi ho sakta. Trade-offs ko explicit karo aur stakeholders ki priority ke mutabiq decide karo.

Part 2: Quality Engineering

2.11 Quality Engineering (QE) kya hai?

Definition: Quality Engineering woh discipline hai jo product ke **poore lifecycle** mein quality ko *build in* karti hai, yaani defects ko sirf dhoondne ke bajaye unhein **rokti** hai.

QE ke 3 principles:

Principle	Matlab	Example
Prevention over detection	Process aur code aise design karo ke defects andar aayein hi nahi	Gari ki regular servicing taake kharab hi na ho, bajaye ke kharab hone ke baad mechanic dhoondna
Whole-team ownership	Developers, testers, ops aur product sab quality ke zimmedar	Restaurant mein khana saaf rakhna sirf health inspector ka kaam nahi, chef, waiter, cleaner sab ka hai
Data-driven improvement	Metrics aur feedback se musalsal behtar hona	Bug count, coverage, customer reviews dekh kar faisla karna, andaze se nahi

2.12 QC vs QA vs QE

	Quality Control (QC)	Quality Assurance (QA)	Quality Engineering (QE)
Focus	Finished product	Process	Poora lifecycle
Goal	Release se pehle defects dhoondna	Defects ko prevent karna	Quality ko continuously build in karna
Methods	Testing, inspection	Standards, audits, reviews	Automation, CI/CD, metrics
Mindset	Reactive (masla hone ke baad react)	Proactive	Engineering-driven

Example (school): QC = exam ke baad papers check karna. QA = padhane ka tareeqa aur syllabus ke standards tay karna. QE = poore saal ka system banana jismein roz ki practice, quizzes, feedback aur automatic progress tracking ho taake fail hone ki naubat hi na aaye.

2.13 Quality Across the Lifecycle (6 stages)

QE har stage par quality activities rakhta hai, aur feedback wapas loop mein jata hai:

#	Stage	Quality activities
01	**Requirements**	Quality goals, acceptance criteria
02	**Design**	Architecture reviews, threat modelling

#	Stage	Quality activities
03	**Build**	Code reviews, static analysis, unit tests
04	**Verify**	Integration tests, performance tests, CI/CD quality gates
05	**Release**	Release criteria
06	**Operate**	Monitoring

Aakhir mein **Feedback and root-cause analysis** wapas pehle stages mein jata hai.

Example: Online food app: requirements mein likha 'order 30 minute mein pahunche' (goal), design mein dekha ke payment API par attack to nahi ho sakta (threat modelling), build mein code review aur unit tests, verify mein load test, release se pehle checklist, aur live hone ke baad monitoring dekhte hain ke koi order fail to nahi ho raha. Jo masla mile uska RCA agle cycle mein jata hai.

Naye terms: *Threat modelling* = design stage par socha jana ke hacker kahan se attack kar sakta hai. *Quality gate* = ek checkpoint, jaise 'agar tests fail hain to code aage nahi jayega'.

2.14 Verification & Validation (Static vs Dynamic)

	Static techniques	Dynamic techniques
Matlab	Code **chalaye baghair**	Code **chala kar**
Examples	Requirements aur design reviews; code inspections aur walkthroughs; static analysis (linters, SonarQube)	Unit aur integration testing; system aur acceptance testing; performance, load aur security testing
Kis se talluq	**Verification:** 'Are we building the product right?'	**Validation:** 'Are we building the right product?'

Example: Imtihan ki copy likhne se pehle outline dekh lena (static, verification) vs copy likh kar paper dena aur teacher se marks lena (dynamic, validation).

2.15 Test Automation Pyramid

Tests ki teen layers, neeche se upar:

Layer	Quantity	Speed / Cost	Kaam
UI / End-to-End (E2E) (sabse upar)	Few	Slow, costly	Critical user journeys ko shuru se akhir tak validate karna
Integration / API (beech mein)	Some	Moderate	Modules aur services ka aapas mein kaam check karna
Unit tests (sabse neeche)	Many	Fast, cheap	Individual functions aur classes ko isolation mein verify karna

Rule of thumb: taqreeban **70% unit, 20% integration, 10% end-to-end** tests.

Kyun pyramid? Neeche wale tests tez aur sasta hain, isliye zyada likhte hain. Upar wale tests slow aur mehnge hain (poora browser chalana padta hai), isliye kam. Agar ulta ho (zyada UI tests, kam unit tests) to test suite slow aur fragile ho jata hai ('ice-cream cone' problem).

Example: Online shop: 'price + tax' ka function unit test se milliseconds mein check hota hai. Cart aur payment ka milna integration test hai. Aur 'login karo, item add karo, order do' ek hi E2E test hai jo bara aur slow hota hai.

2.16 Measuring Quality (Metrics)

'What gets measured gets improved.' Slide ke 6 metrics:

Metric	Matlab	Numerical example
Defect Density	Defects per KLOC (1,000 lines of code)	50 defects in 25,000 lines = 50 ÷ 25 = **2 defects/KLOC**
Defect Removal Efficiency (DRE)	(Release se pehle mile defects ÷ total defects) × 100	Release se pehle 90 mile, baad mein customers ne 10 dhoonde: 90 ÷ 100 × 100 = **90%**
Code Coverage	Automated tests ne code ka kitna % execute kiya	800 lines mein se 600 chali = **75%**
Cyclomatic Complexity	Code mein independent paths ki ginti; kam ho to test karna asaan	Function mein 10 if/else ke raaste = test karne mushkil; chhote functions behtar
MTBF / MTTR	Failures ke beech ka average waqt / theek karne ka average waqt	Availability formula mein use hota hai (2.7 dekho)

Metric	Matlab	Numerical example
Customer Satisfaction	Survey scores aur support tickets: quality jaisi users mehsoos karte hain	App store rating, complaints ki ginti

Yaad rakho: Code coverage 100% ka matlab bug-free nahi hota, sirf yeh ke code chala. Aur cyclomatic complexity jitni *kam*, utna behtar.

2.17 Shift-Left & DevOps

Shift left ka matlab: testing aur quality activities ko timeline mein **left (pehle)** le aana, requirements aur design ke qareeb.

- **Test early:** reviews, TDD (pehle test likho phir code), unit tests.
- **CI/CD quality gates:** Har commit automatically build aur test hota hai.

Shift right ka matlab: release ke *baad* bhi quality dekhna: monitoring, observability, chaos testing (jaan boojh kar system ke hisse tor kar dekhna ke woh sambhalta hai ya nahi).

AI-assisted QE: Test generation, defect prediction, log analysis.

Cost chart (Lecture 2 ki illustrative values): Requirements 1, Design 5, Coding 10, Testing 20, Production 100. *(Dhyan do: Lecture 1 ke chart mein numbers alag the: 1, 3, 7, 15, 30. Dono illustrative hain. Asal point dono mein ek hi hai: jitni der se bug mile, utna mehnga.)*

Example: Imarat banate waqt naksha par ghalti pakad lena aasan hai; chat daalne ke baad deewar girana bohat mehnga.

2.18 Lecture 2 Key Takeaways

- Quality attributes batate hain software kitni achi tarah perform karta hai, unhein **measurable scenarios** ke taur par likho.
- Attributes aapas mein takrate hain; stakeholder ki zaroorat se **prioritise** karo aur trade-offs explicit rakho.
- Quality Engineering lifecycle mein quality **build in** karti hai, sirf aakhir mein nahi.
- **Automate, measure aur shift left** karo taake defects jaldi aur sasta pakde jayein.

Mids ke possible sawal (Lecture 2):

1. Quality attributes aur functional requirements mein farq example ke saath.
2. ISO/IEC 25010 ki 8 characteristics likhein.
3. Runtime aur design-time attributes mein farq batayein.
4. Availability ka formula likhein aur numerical solve karein.
5. Quality Attribute Scenario ke 6 parts example ke saath likhein.

6. Kam az kam 3 attribute trade-offs explain karein.
7. QC, QA aur QE mein farq likhein.
8. Test automation pyramid draw karein aur explain karein.
9. Defect density aur DRE ke formulas aur numericals.
10. Shift-left kya hai? CI/CD ka kya kirdar hai?

Lecture 3: Software Testing: Concepts, Approaches & Test Planning

Teen parts: **Part 1: Testing Concepts**, **Part 2: Testing Approaches**, **Part 3: Test Planning Process**.

Part 1: Testing Concepts

3.1 Software Testing kya hai?

Definition: Software testing kisi system ya uske components ko evaluate karne ka process hai, yeh dekhne ke liye ke woh specified requirements poori karta hai ya nahi, aur users tak pahunchne se pehle **defects dhoondne** ke liye.

Teen baatein:

1. **It is investigative:** Testing masle *dhoondti* hai, sirf yeh confirm nahi karti ke sab theek hai.
2. **It reduces risk:** Release se pehle mila har defect aisa defect hai jo user ne kabhi dekha hi nahi.
3. **It builds confidence:** Achhi tarah test hua system stakeholders ko evidence deta hai ke product use ke qabil hai.

Example: Naya pul (bridge) khulne se pehle us par bhaari gariyan guzar kar test karte hain. Maqsad yeh nahi ke 'pul theek hai' sabit karein, balke yeh dhoondna ke kahan kamzori hai, taake awam ki jaan na jaye.

3.2 Hum test kyun karte hain?

Jitna pehle defect mile, utna sasta fix. Cost chart (Lecture 3):

Stage	Relative cost
Requirements	1
Design	5
Coding	10

Stage	Relative cost
Testing	20
Production	100

Testing se kya milta hai:

- Defects ko tab pakadna jab woh sasta aur aasan hon
- Confirm karna ke system stakeholders ki asal zaroorat poori karta hai
- Faulty release ka business, financial aur safety risk kam karna
- Product quality ka objective, measurable evidence dena

3.3 Error, Defect aur Failure (teeno alag hain!)

Log aksar teeno ko ek hi samajhte hain, magar paper mein farq likhna zaroori hai.

Term	Matlab	Slide ka example
Error (Mistake)	Insan ka ek action jo ghalat natija paida kare	Developer ne requirement ghalat samjhi aur ghalat formula likh diya
Defect (Fault / Bug)	Software ke andar ki kharabi jo error ki wajah se bani	Code mein formula ghalat likha hua hai, trigger hone ka intezar kar raha hai
Failure	System ka asal behaviour expected behaviour se hat jana	User ne calculation chalayi aur screen par ghalat jawab aaya

Chain yaad rakho: Error → Defect → Failure. Insan ghalti karta hai (error), code mein bug ban jata hai (defect), aur jab bug chalta hai to user ko masla nazar aata hai (failure).

Aur example (GPA calculator): Developer ne socha total GPA = total marks ÷ subjects ki ginti (error). Code mein credit hours ka wazan shamil hi nahi (defect). Student ne apna result dekha aur GPA ghalat aaya (failure). Note: bug maujood ho magar woh code path kabhi chale hi na, to failure nahi hoga.

3.4 Verification vs Validation (Lecture 3 ka version)

Verification	Validation
'Are we building the product **right**?'	'Are we building the **right** product?'

Verification	Validation
Check karta hai ke work product (design, code, document) apni specification ke mutabiq hai, zaroori nahi ke software chalaya jaye	Check karta hai ke finished system user ki asal zaroorat poori karta hai, **software chala kar**
Reviews aur inspections, walkthroughs, static analysis	System testing, user acceptance testing (UAT), beta testing

Example: Nikah nama ka form: form mein sab columns sahi bhare hain ya nahi dekhna (verification). Ya mehman ko khana khila kar dekhna ke pasand aaya ya nahi (validation).

3.5 Testing ke 7 Principles (bohat important, har paper mein aata hai)

#	Principle	Matlab	Example
1	**Testing shows presence of defects**	Testing sabit kar sakti hai ke defects hain, yeh nahi ke software bilkul defect-free hai	100 tests pass hone ka matlab yeh nahi ke 101wa test pass hoga
2	**Exhaustive testing is impossible**	Har input aur combination test karna sirf bohat chhote systems ke liye mumkin	10 digit ka phone number field: 10 arab combinations. Sab test nahi ho sakte, isliye techniques use karte hain (EP, BVA)
3	**Early testing saves time & money**	Lifecycle mein jaldi shuru karo, defects tab pakdo jab sasta ho	Requirements review mein hi ghalti pakad lena
4	**Defects cluster together**	Chhote se modules mein zyada tar defects hote hain (Pareto: ~80% bugs ~20% modules mein)	Payment module mein sab se zyada bugs, to wahan zyada testing
5	**Beware the pesticide paradox**	Wohi tests baar baar chalane se naye bugs milna band ho jata hai; tests badalte raho	Machhar maarne ki wohi dawai kuch arse baad asar nahi karti, jaise machhar resistant ho jate hain
6	**Testing is context dependent**	Alag systems alag tareeqe aur rigor se test hote hain	Banking system ko game se zyada sakht testing chahiye

#	Principle	Matlab	Example
7	**Absence-of-errors is a fallacy**	Koi defect na milna ka matlab yeh nahi ke system kaam ka hai; shayad woh user ki asal zaroorat hi poori nahi karta	Bug-free app jo koi use hi nahi karna chahta

3.6 Levels of Testing (4 levels)

Level	Kis cheez ko test karta hai	Owner	Example
Unit Testing	Individual functions/classes isolation mein	Developer	Login function ka password-check
Integration Testing	Combined modules ka interaction	Developer / Tester	Login module aur database ka milna
System Testing	Poori integrated application	Test team	Pehle se akhir tak poora app chalana
Acceptance Testing	Business needs, user ke nazariye se	Users / Client	Client app dekh kar 'haan yehi chahiye' kehta hai

Jaise jaise system assemble hota hai, testing ka scope barhta jata hai.

Example (gari): Pehle har piston/spark plug alag test (unit), phir engine ke parts jodh kar test (integration), phir poori gari road test (system), aur aakhir mein customer test drive kar ke qabool karta hai (acceptance).

Part 2: Testing Approaches

3.7 Static vs Dynamic Testing

	Static Testing	Dynamic Testing
Code	**Execute NAHI** hota	**Execute** hota hai
Matlab	Documents, designs aur source code mein defects *software chalne se pehle* dhoondta hai	Software ko real ya simulated input de kar uska actual behaviour dekhta hai

	Static Testing	Dynamic Testing
Examples	Requirements aur design reviews; code walkthroughs aur inspections; static analysis tools (linters)	Unit, integration aur system testing; performance aur load testing; exploratory aur usability testing

Example: Static = Gari kharidne se pehle uske papers aur engine number check karna. Dynamic = Gari chala kar dekhna ke brake, steering aur engine kaisa kaam karte hain.

3.8 Black-Box, White-Box aur Grey-Box Testing

	Black-Box	White-Box	Grey-Box
Matlab	Internal code ka ilm **nahi**; sirf inputs aur expected outputs	Internal logic, code paths aur structures **seedha** test hote hain; source code ka ilm zaroori	Dono ka mix: internals ka thoda ilm istemal kar ke behtar black-box tests design karna
Technique examples	Equivalence partitioning, boundary value analysis	Statement coverage, branch/path coverage	Database schema ka kuch ilm rakh kar API test karna
Kaun karta hai (aam taur par)	Tester	Developer	Tester jise kuch technical ilm ho

Example (TV remote): Black-box: aap button dabate ho aur dekhte ho ke channel badla ya nahi, andar ka circuit nahi dekhte. White-box: engineer circuit kholkar har wire aur chip ka raasta check karta hai. Grey-box: aap ko pata hai ke remote ke andar infrared signal bhejta hai, to aap us ko dhyan mein rakh kar test banate hain (jaise dhoop mein ya andhere mein remote chalana).

3.9 Functional vs Non-Functional Testing

Functional Testing (WHAT)	**Non-Functional Testing** (HOW WELL)
Check karta hai system **kya** karta hai: har feature requirements ke mutabiq chalta hai ya nahi	Check karta hai system woh kaam **kitni achi tarah** karta hai: functionality ke ilawa quality attributes
Unit testing: individual functions	**Performance testing:** load mein speed
Integration testing: combined modules	**Security testing:** attack ke khilaf mazahmat

Functional Testing (WHAT)	**Non-Functional Testing** (HOW WELL)
Smoke testing: core functions kaam karte hain	**Usability testing:** istemal mein asani
Regression testing: kuch toota to nahi	**Compatibility testing:** alag environments mein

Example (online shop): Functional = 'Add to cart' button par click karne se item cart mein aata hai. Non-functional = Jab 10,000 log ek saath button dabayein to bhi 2 second mein item add ho; ya mobile aur laptop dono par theek chale.

3.10 Manual vs Automated Testing

	Manual Testing	**Automated Testing**
Kaise	Insan test cases ko step by step bina tool ki madad ke chalata hai	Scripts aur tools khud tests chala kar results compare karte hain
Kahan best	Exploratory, usability aur one-off testing	Wohi tests jo baar baar hon, jaise regression suites
Fayda	Flexible, scripting ki zaroorat nahi	Tez aur consistent (ek jaisa)
Nuqsan	Regression ke liye slow aur boring	Shuru mein build aur maintain karne ka bara investment
Kab use karein	New features, UX evaluation, ad-hoc exploration	Regression, load testing, repetitive checks

Example: Naye app design par 'yeh button dekhne mein kaisa lag raha hai' kisi insan ko hi dekhna padega (manual). Lekin har release par 500 purane tests dobara chalane ho to script behtar hai (automated). Jaise roz subah 500 gharon mein khud jaa kar bijli dekhne ke bajaye ek meter-reading system laga lena.

3.11 Equivalence Partitioning (EP) aur Boundary Value Analysis (BVA)

Yeh dono black-box techniques hain jo 'exhaustive testing impossible hai' wale principle ka hal hain.

Example field: Age field jo sirf **18 se 60** tak accept karti hai.

Equivalence Partitioning

Input data ko aise **partitions** (groups) mein baanto jahan system ka behaviour same hona expected hai, phir har partition se **sirf ek representative value** test karo, har possible input nahi.

Partition	Range	Test value (example)	Expected
Invalid (neeche)	18 se kam	10	Reject
Valid	18 se 60	30	Accept
Invalid (upar)	60 se zyada	70	Reject

Logic: Agar 30 accept hota hai to 31, 40, 55 bhi hone chahiye; agar 10 reject hota hai to 5, 12, 17 bhi reject hone chahiye. Isliye 43 values ke bajaye sirf 3 test.

Boundary Value Analysis (BVA)

Defects aksar valid range ke **kinaron (edges)** par hote hain, isliye har boundary ke **ussi par, ek neeche aur ek upar** wali values test karte hain.

Test values: 17, 18, 19 ... 59, 60, 61 (yeh 6 values).

Value	Expected
17	Reject (boundary se ek neeche)
18	Accept (lower boundary)
19	Accept
59	Accept
60	Accept (upper boundary)
61	Reject (boundary se ek upar)

Kyun zaroori? Yeh 6 values **off-by-one errors** pakadti hain jo EP akeli miss kar sakti hai. Jaise developer ne galti se `age > 18` likh diya jab `age >= 18` chahiye tha; EP ki value 30 yeh bug nahi pakdegi, magar BVA ki value 18 foran pakad legi.

Yaad rakho: EP = har group se ek value. BVA = kinaron ki values. Dono saath use karo.

3.12 Decision Table aur State Transition Testing

Decision Table Testing

Jab system ka behaviour **conditions ke combination** par depend kare, tab har true/false combination ko list kar ke har ek ka result likhte hain.

Slide ka example (discount):

Has coupon	Order > $50	Discount
Yes	Yes	20%
Yes	No	10%
No	Yes	5%
No	No	0%

2 conditions × 2 values (yes/no) = **4 combinations**, har ek ke liye ek test case. Formula: n conditions ke liye 2 ki power n combinations (3 conditions = 8).

Real example: Food app: 'coupon hai' aur 'order 1500 se zyada' dono sahi hon to maximum discount, dono ghalat hon to koi discount nahi. Decision table se koi combination test karna bhoolta nahi.

State Transition Testing

Jab system alag **states** mein ho sakta ho aur **events** par ek state se doosri mein jata ho. Tests **valid transitions** cover karte hain **aur yeh bhi check karte hain ke invalid transitions reject hon.**

Slide ka example (order): Pending → Approved → Shipped, ya Pending → Rejected. Test yeh check karega ke **Rejected order kabhi Shipped nahi ho sakta.**

Current state	Event	Next state	Valid?
Pending	Manager approves	Approved	Valid
Approved	Courier picks up	Shipped	Valid
Pending	Manager rejects	Rejected	Valid
Rejected	Courier picks up	Shipped	**Invalid, reject hona chahiye**

Example: ATM card: Active → Blocked (3 ghalat PIN ke baad). Blocked card se paisa nahi nikalna chahiye. Yeh invalid transition ka test hai.

3.13 Regression, Smoke aur Sanity Testing

	Regression	Smoke	Sanity
Matlab	Purane existing tests dobara chala kar confirm karna ke jo pehle chalta tha woh toota nahi	Naye build par jaldi se broad check ke sabse zaroori functions chalte bhi hain ya nahi	Narrow, focused check ke ek specific fix ya feature ne apna kaam kiya
Kab	Har code change ya bug fix ke baad	Har naye build ke fauran baad	Ek specific bug fix ke baad, deeper testing se pehle
Scope	Bara	Broad magar shallow	Narrow aur deep

Example: Daftar mein naya server install hua (naya build). **Smoke:** pehle dekho ke server on hota hai, login hota hai, internet chalta hai (kuch bhi nahi chala to aage test karne ka faida nahi). **Sanity:** Printer ka masla theek kiya gaya to sirf yeh dekho ke printer ab print kar raha hai. **Regression:** Printer theek karne ke baad check karo ke scanner, internet aur baqi cheezein ab bhi sahi chal rahi hain.

Smoke ko aksar 'build verification test' bhi kehte hain. *(Extra point, slides mein nahi.)*

Part 3: Test Planning Process

3.14 Test Plan kya hai?

Definition: Test plan ek **formal document** hai jo project ki testing activities ka **scope, objectives, approach, resources, schedule aur risks** describe karta hai. Yeh woh blueprint hai jo 'hum is system ko test karenge' ko ek concrete, manageable effort bana deta hai.

Fayde (3):

1. **Sets clear objectives:** Sab ko pata hota hai 'done' aur 'quality' ka is project mein kya matlab hai.
2. **Aligns stakeholders:** Managers, developers aur testers ki expectations ek jaisi hoti hain.
3. **Manages risk:** Risks costly problems ban'ne se pehle pehchane aur handle kiye jate hain.

Example: Shaadi ka plan: kitne mehmaan (scope), kya menu (approach), kaun kya zimmedari sambhale (roles), kab kya hoga (schedule), baarish ho gayi to kya karenge (risk). Bina plan ke shaadi afra-tafri mein badal jati hai, testing bhi.

3.15 Components of a Test Plan (IEEE 829 par based)

Component	Matlab	Example
Objectives & Scope	Kya test hoga, aur utna hi zaroori, kya **nahi** hoga	Login aur checkout test honge; admin dashboard is release mein nahi
Test Strategy / Approach	Kaun se levels aur techniques (black-box, regression, automation)	Unit tests developers; E2E checkout automated
Resources & Roles	Kaun kya karega: testers, environments, tools, test data	2 testers, 1 staging server, Jira
Schedule & Milestones	Har test activity ki start/end dates, project timeline ke mutabiq	Test design: 1-5 Nov; execution: 6-15 Nov
Risks & Contingencies	Testing effort ke known risks aur unka hal	Risk: tester beemar ho jaye. Plan: cross-trained backup tester
Entry & Exit Criteria	Testing shuru karne ki shartein aur testing mukammal maanne ki shartein	Entry: build stable aur smoke pass. Exit: 95% tests pass, koi critical defect open nahi

3.16 Test Planning Process: 6 Steps

Step	Kaam	Matlab
1	**Analyse requirements**	Samjho kya test karna hai aur testable requirements pehchano
2	**Define test strategy**	Tay karo kaun se levels, types aur techniques use honge
3	**Define objectives**	Saaf, measurable goals likho ke testing se kya hasil karna hai
4	**Plan resources**	Log, environments, tools aur test data pehchano
5	**Build the schedule**	Effort estimate karo aur realistic timeline set karo
6	**Set entry/exit criteria**	Tay karo testing kab shuru ho sakti hai aur kab mukammal maani jayegi

(Dhyan: slide mein order bilkul yehi hai; paper mein isi order mein likhna.)

3.17 Test Estimation aur Team Roles

Test effort estimate karne ke tareeqe

Schedule banane se pehle andaza lagana padta hai ke kitna testing kaam hai:

1. **Work Breakdown Structure (WBS):** Testing ko chhote, estimate-qabil tasks mein torna. *Example:* 'Login test' = test cases likhna (4 ghante) + chalana (2 ghante) + bug report (1 ghanta).
2. **Historical / experience-based data:** Pichhle milte julte projects ki effort use karna. *Example:* Pichhli app ke 20 screens ko 10 din lage the, to nayi app ke 30 screens ko ~15 din.
3. **Function Point / Test Point Analysis:** Functionality ke size se estimate karna.
4. **Expert judgement:** Tajarbakaar testers seedha andaza dein.

Test team ke roles

Role	Kaam
Test Manager	Test plan, schedule ka malik; stakeholders ko progress report karta hai
Test Lead	Test strategy design karta hai aur roz ki test activities coordinate karta hai
Test Engineer / Tester	Test cases likhta aur chalata hai, aur milne wale defects log karta hai
Automation Engineer	Automated test scripts aur frameworks banata aur maintain karta hai

Example (cricket team): Test Manager = team manager (planning aur board se baat), Test Lead = captain (match ka strategy aur field par coordination), Testers = khiladi jo asal kaam karte hain, Automation Engineer = woh jo bowling machine aur analysis software chalata hai.

3.18 Lecture 3 Key Takeaways

- Testing software ko evaluate kar ke defects dhoondne aur confidence banane ka process hai. **Error, defect aur failure teen alag cheezein hain.**
- Koi ek approach kaafi nahi: static aur dynamic, black-box aur white-box, manual aur automated, sab ki jagah hai.

- EP, BVA aur decision tables jaisi techniques testing ko **systematic** banati hain, random nahi.
- Ek solid test plan (scope, strategy, resources, schedule, risks, exit criteria) testing ko guesswork se **managed process** bana deta hai.

Mids ke possible sawal (Lecture 3):

1. Error, defect aur failure mein farq example ke saath.
2. Testing ke 7 principles explain karein.
3. Testing ke 4 levels aur unke owners likhein.
4. Static vs dynamic testing; black-box vs white-box.
5. Age field (18-60) ke liye EP aur BVA test cases banayein.
6. Decision table banayein (coupon aur order amount wala).
7. Regression, smoke aur sanity testing mein farq.
8. Test plan kya hai? Iske components likhein.
9. Test planning ke 6 steps likhein.
10. Test team ke roles aur effort estimation ke tareeqe.

Lecture 4: Testing Documentation, Reporting & Historical Data Recording

(Course content: Software Testing Lifecycle. Is lecture ki file ka naam SQL_L_4 hai, magar content SQE ka Lecture 4 hai.)

Learning objectives: (1) Testing documentation ki zaroorat samjhana, (2) main test documents aur unke contents pehchanna, (3) test case aur defect report likhna, (4) test reporting aur uske metrics describe karna, (5) historical data kaise record aur dobara use hota hai.

4.1 Testing ko document kyun karte hain? (4 wajahein)

Wajah	Matlab	Example
Communication	Team aur client ke saath test status share hota hai	Manager ko poochna na pade 'testing kahan tak pahunchi?', report dekh le
Repeatability	Koi bhi wohi test dobara chala sakta hai	Naya tester purane test case ke steps follow kar ke wohi result le sakta hai

Wajah	Matlab	Example
Traceability	Har test ek requirement se juda hota hai	Kisi requirement ka koi test na ho to foran pata chal jata hai
Evidence	Audits aur accreditation ke liye saboot	Client ya auditor poochay 'yeh feature test hua?' to saboot dikha sako

Teacher ka sawal (slide notes se): Agar koi tester chhor jaye aur kuch likha hi na ho to kya hoga? Jawab: sab knowledge us ke saath chali jati hai, naya banda scratch se shuru karega, aur pata hi nahi hoga ke kya test ho chuka. **Example:** Restaurant ka chef bina recipe likhe chala jaye to koi usi zaiqay ka khana dobara nahi bana sakta.

4.2 Types of Test Documents (7)

Yeh **IEEE 829** style ki documentation hai, jo ab **ISO/IEC/IEEE 29119-3** mein cover hoti hai.

#	Document	Purpose
1	**Test plan**	Kya, kaise, kaun aur kab (what, how, who, when)
2	**Test design specification**	Kaun se features aur kaun si techniques use hongi
3	**Test case specification**	Inputs, steps, expected results
4	**Test procedure**	Test cases kis order mein chalenge
5	**Test log**	Jo execute hua uska record
6	**Defect report**	Ek failure ki tafseel
7	**Test summary report**	Final results aur conclusions

Yaad karne ka trick: Plan → Design → Case → Procedure → Log → Defect → Summary (shuru se aakhir tak ka safar).

Example (safar): Plan = safar ka program (kahan, kab, kaun jayega). Design = route aur gari ka intikhab. Case = har mod par kya karna hai. Procedure = kis order mein jana hai. Log = safar mein kahan kab pahunche. Defect report = gari kharab hui to uski detail. Summary = wapas aa kar safar ka khulasa.

4.3 Test Plan (Lecture 4 ke mutabiq)

Test plan testing shuru hone se **pehle** likha jata hai aur scope badalne par **update** hota hai.

Main contents	Also include
Scope aur objectives	Schedule aur resources
Test items aur features	Roles aur responsibilities
Approach aur test levels	Risks aur contingencies
Environment aur tools	Deliverables aur approvals
Entry aur exit criteria	

(Lecture 3 mein bhi test plan ke components aaye the. Dono ko mila kar yaad rakho.)

4.4 Test Design aur Test Case Specification

- **Test design specification:** Test karne ke features list karti hai, chuni hui **technique** (EP, BVA, decision table) aur **pass/fail criteria**.
- **Test case specification:** Har case ko **ID, preconditions, inputs, expected output aur dependencies** deti hai.

Traceability (important): Har test case ek **requirement ID** se link hota hai, taake jo requirement test hi nahi hui woh aasani se pakdi jaye. Is ko **requirement-to-test-case mapping** (traceability matrix) kehte hain.

Requirement ID	Requirement	Test cases
REQ-LOGIN-01	Valid user login kar sake	TC-01
REQ-LOGIN-02	Ghalat password par error dikhe	TC-02, TC-03
REQ-LOGIN-03	Khali fields par required-field message	TC-06
REQ-LOGIN-04	Password reset	*Koi test case nahi, coverage gap!*

Yeh table dekh kar foran pata chal jata hai ke REQ-LOGIN-04 test hi nahi hui.

4.5 The Test Case Document

Slide ka example (SauceDemo login website, lab ka running example):

Field	Example
Test case ID	TC-01
Requirement ID	REQ-LOGIN-01

Field	Example
Precondition	User registered hai
Steps	Valid email aur password daalo, Login par click karo
Test data	standard_user aur valid password
Expected result	Products page khulta hai
Actual and status	Products page khula, **Pass**

Har field ka kaam: *ID* test ko pehchanne ke liye; *Requirement ID* traceability ke liye; *Precondition* test se pehle kya hona chahiye; *Steps* kaise chalana hai; *Test data* kaun si values; *Expected* kya hona chahiye; *Actual* asal mein kya hua; *Status* Pass ya Fail.

Fail wala example:

Field	Example
Test case ID	TC-06
Requirement ID	REQ-LOGIN-03
Precondition	Login page khula hai
Steps	Dono fields khali chhoro, Login par click karo
Expected result	'Required field' message dikhe
Actual and status	Page silently reload ho gaya, **Fail** (Defect D-014)

4.6 Test Procedure, Test Data aur Test Scripts

- **Test procedure:** Execution ka step by step **order**, jismein **setup aur cleanup** bhi shamil hain.
- **Test data: Valid, invalid aur boundary values** jo pehle se tayyar hon aur test cases se **alag** rakhi jayein.
- **Test scripts:** Manual steps ka **automated version** (aage ke labs mein aayega).

Teacher ka point: Test data documentation ka hissa hai, baad ki socha hua kaam nahi.

Example: Login test se pehle test account banana (setup), tests chalana, aur aakhir mein test account delete karna (cleanup) procedure mein likha jata hai. Test data jaise `standard_user` (valid), `locked_out_user` (invalid), aur 1 character ka username (boundary) alag file mein rakhte hain taake test cases aur data dono alag alag update ho saken.

4.7 Test Log: Execution ka Record

Log ek **time-ordered history** deta hai: kya chalaya gaya, kis ne chalaya, aur kis build par.

Date	Tester	Test case	Build	Result	Remarks
20 Oct	Tester A	TC-01	v1.2	Pass	None
20 Oct	Tester A	TC-06	v1.2	Fail	Defect D-014
21 Oct	Tester B	TC-06	v1.3	Pass	Retest after fix

Kahani samjho: TC-06 v1.2 par fail hua aur defect D-014 bana. Developer ne fix kiya aur nayi build v1.3 aayi. Tester B ne retest kiya to pass, yaani defect theek ho gaya.

Example: Hostel ya school ki attendance/registry jaisa: kis din, kis ne, kya kiya, sab likha hota hai.

4.8 The Defect Report

Slide ka example:

Field	Example
Defect ID	D-014
Title	Khali login par error nahi dikhta
Steps to reproduce	Page kholo, fields khali chhoro, Login par click karo
Expected result	Required-field message
Actual result	Page silently reload ho jata hai
Severity and priority	Medium aur High
Status	New

Acchi defect report ka usool (slide note): Developer bina tester se kuch poochay bug **reproduce** kar sake. Isliye title saaf, steps exact aur expected/actual alag alag likho.

Bura vs achha title: 'Login kaam nahi kar raha' (bura, mubham) vs 'Khali fields par Login dabane se error message nahi aata' (achha).

4.9 Defect Life Cycle

New → Assigned → Fixed → Retested → Closed

State	Matlab
New	Tester ne defect log kiya
Assigned	Developer ko fix karne ke liye diya gaya
Fixed	Developer ne fix kar diya
Retested	Tester ne dobara check kiya
Closed	Retest pass, defect band
Reopened	Retest **fail** hua, defect wapas **Assigned** par jata hai
Rejected	Defect valid nahi (jaise bug hai hi nahi, ya expected behaviour hai)
Deferred	Abhi fix nahi hoga, aage kisi release mein

(Lecture 1 mein yehi lifecycle 'New, Assigned, Fixed, Verified, Closed' likha tha. Verified aur Retested ek hi step hain; paper mein jo wording teacher ki slide mein ho wohi likho.)

Example: Tester ne bug dala (New) → manager ne developer Ali ko diya (Assigned) → Ali ne theek kiya (Fixed) → tester ne dobara try kiya (Retested) → theek mila to Closed; agar phir bhi masla hai to Reopened. Agar developer kahe 'yeh bug nahi, design ke mutabiq hai' aur manager maan jaye to Rejected. Agar masla chhota hai aur release kal hai to Deferred.

4.10 Severity vs Priority (bohat common sawal)

	Severity	Priority
Matlab	System par **impact** (kitna bura)	Fix karne ki **urgency** (kitni jaldi)
Aam taur par kaun set karta hai	Tester	Manager ya client
High severity, low priority	App ek kam istemal hone wali report mein crash hoti hai	Agli release mein fix karo
Low severity, high priority	Home page par company ka naam ghalat	Foran fix karo

Yaad rakho: Severity = bug kitna **khatarnak**. Priority = kitni **jaldi** theek karna hai.

Aur examples:

- *High severity + high priority:* Payment page par app crash. Foran theek karo.

- *Low severity + low priority:* Kisi aise page par chhota spelling mistake jise koi nahi dekhta.
- *Low severity + high priority:* CEO ke naam ki spelling ghalat website par, kyunke company ki izzat ka sawal hai.

4.11 Test Reporting: Teen Qism ki Reports

Report	Kis ke liye	Kab
Defect report	Developers	Failure milte hi, **ek failure ki tafseel**
Progress report	Managers	Daily ya weekly status: executed, passed, failed aur **blocked**
Summary report	Clients aur management	Aakhir mein: final results, open defects aur **release recommendation**

Har report ka audience alag hai: developers, managers aur clients.

Blocked ka matlab: test chalaya nahi ja sakta kyunke koi aur cheez rukavat hai (jaise login hi nahi chal raha to uske baad ke tests blocked).

4.12 Summary Report aur Metrics

Report mein kya hota hai: Scope tested aur environment; pass/fail counts; severity ke hisab se open defects; risks aur recommendation.

Common metrics (4):

Metric	Formula	Numerical example
Pass rate	Passed ÷ Executed	40 chalaye, 34 pass: 34÷40 = **85%**
Execution rate	Executed ÷ Planned	50 planned, 40 chalaye: 40÷50 = **80%**
Defect density	Defects ÷ Size	12 defects, 4 KLOC: 12÷4 = **3 defects/KLOC**
Removal efficiency	Release se pehle mile ÷ Total	45 pehle mile, 5 baad mein: 45÷50 = **90%**

Dhyan: Pass rate ka denominator *executed* hai, *planned* nahi. Execution rate mein denominator *planned* hai.

Example recommendation: 'Critical defects zero hain, 2 medium open hain, pass rate 95%. Release recommended.' Ya 'ek critical defect open hai, release na karein.'

4.13 Historical Data Recording

Historical data testing process ki **yaaddasht (memory)** hai.

Data recorded	Example
Defects found per module	Login 6, Cart 3
Effort spent	Test design par 40 ghante
Results per build	v1.2: 90% pass
Defect causes	Input validation ki kami
Test cases reused	150 mein se 120

4.14 Historical Data ka Istemal

Use	Matlab	Example
Estimation	Agle project ka time aur effort pichhli effort se plan karo	Pichhle project mein test design par 40 ghante lage the, to similar project mein ~40 ghante rakho
Defect trends	Kamzor modules pehchano aur wahan zyada testing karo	Pichhli release mein Login mein 6 bugs aaye the, to is dafa Login par zyada tests
Process improvement	Common causes dhoondo aur aage rok do	Agar zyada bugs 'input validation missing' se aaye, to validation checklist banao

Good practice: Consistent record rakho, **ek hi jagah** store karo, aur **har release ke baad review** karo.

Example (cricket): Coach purane matches ka data dekh kar jaanta hai ke opposition ka kaun sa bowler kamzor hai aur kis ground par kya strategy chali thi. Wohi data agle match ki planning mein kaam aata hai.

4.15 Lecture 4 Summary aur Review Questions

Key points: Communicate aur repeat karne ke liye document karo; Plan, design, case, log, defect, summary; reports ke audiences alag hain; history se agle projects behtar hote hain.

Review questions (model jawab):

1. **Chaar test documents ke naam:** Test plan, test case specification, test log, defect report (ya test summary report, test design specification, test procedure).
2. **Defect report ke paanch fields:** Defect ID, title, steps to reproduce, expected result, actual result (ya severity/priority, status).
3. **Severity vs priority:** Severity = system par impact (tester set karta hai); Priority = fix ki urgency (manager ya client). Example: crash in rare report = high severity, low priority; wrong company name = low severity, high priority.
4. **Historical data kyun rakhte hain:** Estimation behtar hoti hai, kamzor modules samne aate hain, aur common causes dhoond kar process improve hota hai. (Yeh testing process ki memory hai.)

Mids ke possible sawal (Lecture 4):

1. Testing documentation kyun zaroori hai? (4 wajahein)
2. Test documents ki 7 types likhein.
3. Ek test case banayein (login ka) saari fields ke saath.
4. Ek defect report likhein jismein saari fields hon.
5. Defect life cycle draw karein aur Reopened/Rejected/Deferred samjhayein.
6. Severity aur priority ki 4 combinations examples ke saath.
7. Test reports ki teen qismein aur unke audience.
8. Pass rate, execution rate, defect density, DRE ke formulas aur numericals.
9. Historical data kya hai aur kaise use hota hai?
10. Traceability kya hai aur kyun zaroori?

Quick Revision Sheet (Paper se pehle raat ko yeh parhna)

Sab Formulas

Formula	Likhna kaise hai
Availability	MTBF ÷ (MTBF + MTTR)
Defect Density	Defects ÷ KLOC
Defect Removal Efficiency	(Release se pehle mile defects ÷ total defects) × 100
Pass rate	Passed ÷ Executed
Execution rate	Executed ÷ Planned
Test pyramid	~70% unit, 20% integration, 10% E2E

Mukhtasar Farq (One-liners)

- **QA vs QC:** QA = process, prevention, poore lifecycle. QC = product, detection, testing ke dauran/baad.
- **QE:** QA + QC se bara: lifecycle mein quality build in (automation, CI/CD, metrics).
- **Verification vs Validation:** Product **right** bana? vs **right** product bana?
- **Error / Defect / Failure:** insan ki ghalti / code mein bug / user ko nazar aane wala galat behaviour.
- **Static vs Dynamic:** code chalaye baghair / code chala kar.
- **Black / White / Grey box:** code ka ilm nahi / ilm hai / thoda ilm.
- **Functional vs Non-functional:** KYA karta hai / KITNI ACHI TARAH karta hai.
- **Regression / Smoke / Sanity:** purane tests dobara / naye build ka jaldi broad check / ek fix ka narrow check.
- **Severity vs Priority:** kitna bura / kitni jaldi.
- **Prevention / Reduction / Containment:** rokna / dhoondna aur hatana / nuqsan limit karna.
- **Design quality vs Conformance quality:** naksha achha / naksha ke mutabiq bana.

Important Lists (ginti yaad rakho)

- **6 quality dimensions (L1):** Functionality, Reliability, Usability, Efficiency, Maintainability, Portability.

- **8 ISO 25010 characteristics (L2):** Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability.
- **5 QA process steps (L1):** Define standards, Train, Review, Audit, Improve.
- **4 RCA steps (L1):** Identify, Trace to origin, Categorize cause, Correct and monitor.
- **5 testing levels (L1):** Unit, Integration, System, Acceptance, Regression. *(L3 mein 4 levels: Unit, Integration, System, Acceptance.)*
- **6 QA scenario parts (L2):** Source, Stimulus, Artifact, Environment, Response, Response Measure.
- **6 lifecycle stages (L2):** Requirements, Design, Build, Verify, Release, Operate.
- **7 testing principles (L3):** Presence of defects; Exhaustive impossible; Early testing; Defects cluster; Pesticide paradox; Context dependent; Absence-of-errors fallacy.
- **6 test plan components (L3):** Objectives and scope; Strategy; Resources and roles; Schedule; Risks; Entry/exit criteria.
- **6 test planning steps (L3):** Analyse requirements, Define strategy, Define objectives, Plan resources, Build schedule, Set entry/exit criteria.
- **7 test documents (L4):** Test plan, design spec, case spec, procedure, log, defect report, summary report.
- **Defect life cycle (L4):** New, Assigned, Fixed, Retested, Closed (+ Reopened, Rejected, Deferred).

Mids ke Tips

1. Har answer mein pehle **definition**, phir **table ya points**, phir **ek example** likho. Examiner ko example se pata chalta hai ke tum ne concept samjha hai, ratta nahi lagaya.
2. Jahan 'farq likhein' aaye wahan **table** banao.
3. Numerical ke liye formula pehle likho, phir values rakho, phir answer.
4. Cost-of-defect chart ke numbers dono lectures mein alag hain (1, 3, 7, 15, 30 aur 1, 5, 10, 20, 100), is liye question ke saath wala chart/number use karo.
5. Slide mein jo exact words hon (jaise 'Are we building the product right?') woh as-is likhna behtar hai.

Allah kare mids mein behtareen marks aayein, Moto! Agar kisi topic ko aur asaan chahiye ya practice questions aur MCQs chahiye to bata dena.